Worksheet — 2.1 Elements of Computational Thinking

Name: __ **Date:** ____

OCR A Level Computer Science (H446) — Component 02 — Spec ref 2.1.1–2.1.5

Time: ~30 min.

> Reminder: this topic is examined by **application to a scenario**, not by reciting definitions. Always name the scenario's *actual* inputs, conditions and sub-procedures.

The scenario — *BookIt* library room-booking system

Throughout this worksheet you will work on one scenario: a small program called **BookIt** that lets students book a study room in a library. A student picks a room and a time slot, and the system either confirms the booking or rejects it. It shows the rooms that are free, takes the student's ID and chosen slot, checks availability, then stores the booking and prints a confirmation.

Keep this scenario in mind for every task below.

Task 1 — Key-term match (the five thinking skills)

Draw a line (or write the matching letter) to connect each thinking skill to its definition.

Thinking skill		Definition
1. Thinking abstractly		A. Identifying decision points where the flow of control branches on a condition.
2. Thinking ahead		B. Spotting parts of a problem that can run at the same time.
3. Thinking procedurally		C. Removing/hiding unnecessary detail so only relevant information remains.
4. Thinking logically		D. Breaking a problem into an ordered sequence of steps and sub-procedures.
5. Thinking concurrently		E. Planning before coding: inputs/outputs, preconditions, caching, reusable components.

Answers: 1 ___ 2 ___ 3 ___ 4 ___ 5 ___

Task 2 — Thinking ahead: inputs, outputs and preconditions (BookIt)

(a) List **three inputs** the BookIt system needs from the user.

1.
2.

3.

(b) List **two outputs** the system produces.

1.

2.

(c) State **one precondition** that must be true *before* a booking can be confirmed.

> Tip: "Validate the student ID" is a **process**, not an input or output — don't list it in (a) or (b).

Task 3 — Abstraction: keep or remove? (BookIt)

You are modelling each *room* in BookIt. For each piece of detail below, tick whether you would **KEEP** it in the model or **REMOVE** it, and write a one-line reason for the first three.

Detail about a room	KEEP	REMOVE
Room number / ID	☐	☐
Maximum capacity (seats)	☐	☐
Colour of the carpet	☐	☐
Which time slots are already booked	☐	☐
The make of chairs in the room	☐	☐

Reasons:

KEEP example — _____

REMOVE example — _____

Why is the removed detail irrelevant **to this problem** specifically? _____

Task 4 — Decomposition: break it into sub-problems (BookIt)

Decompose the "make a booking" task into **four to six** sub-problems / sub-procedures. Then number them in the order they must run.

Order	Sub-procedure
____	_____
____	_____
____	_____
____	_____
____	_____
____	_____

Which sub-procedure **depends on the output** of an earlier one? State which depends on which.

Task 5 — Thinking logically: decision points (BookIt)

Identify **two decision points** in BookIt. For each, write the **exact Boolean condition** and state what happens in **both** branches.

Decision point 1

Condition (Boolean): `IF _____`

If TRUE: _____

If FALSE: _____

Decision point 2

Condition (Boolean): `IF _____`

If TRUE: _____

If FALSE: _____

Task 6 — Thinking concurrently: what can run at once? (BookIt)

(a) Name **two processes** in BookIt that could run **concurrently** because they have **no data dependency** between them.

1.
2.

(b) Name **two steps that must stay sequential** (one depends on the other) and say why.

(c) State **one benefit** and **one trade-off** of running parts of BookIt concurrently.

Benefit: _____

Trade-off: _____

Task 7 — Abstraction vs decomposition: which is which?

For each statement, write **A** (abstraction) or **D** (decomposition).

1. Ignoring the carpet colour when modelling a room. ____
2. Splitting "make a booking" into check-availability, store-booking and confirm. ____
3. Representing the whole library as a list of rooms with free/busy flags. ____
4. Breaking the program into separate display, validate and save procedures. ____
5. Treating "savings" and "current" accounts as one general `Account` model. ____
6. Dividing the timetable problem into "morning slots" and "afternoon slots" handlers. ____

Challenge box

> **Caching in BookIt.** The "rooms currently free" list is requested by many students at once and is expensive to recompute from the database every time. Explain how **caching** this list would help, and give **one trade-off**, including what could make the cached data go **stale** and how you would handle it.